

온라인 쇼핑몰 ERD 상세 설계서 v1

0. 설계 목적

이 문서는 Spring Boot 기반 온라인 쇼핑몰의 운영 가능한 데이터 모델을 정의한다.

핵심 목표는 다음과 같다.

1. 주문·결제·재고의 정합성을 지킨다.
2. 상품, 주문, 결제, 배송, 리뷰의 상태 변경 이력을 추적할 수 있게 한다.
3. 동시 주문, 중복 결제, 중복 리뷰 같은 Race Condition을 DB 제약조건으로 방어한다.
4. 조회 API 패턴에 맞는 인덱스를 설계한다.
5. 배치, 장애 복구, 운영 모니터링을 위한 테이블을 포함한다.

1. 전체 ERD 개요

```
erDiagram
    MEMBER ||--o{ ORDERS : places
    MEMBER ||--o{ REVIEW : writes
    MEMBER ||--o{ CART : owns
    MEMBER ||--o{ COUPON_ISSUE : receives
    MEMBER ||--o{ POINT_HISTORY : has

    CATEGORY ||--o{ PRODUCT : contains
    PRODUCT ||--|| INVENTORY : has
    PRODUCT ||--o{ ORDER_ITEM : ordered_as
    PRODUCT ||--o{ REVIEW : reviewed
    PRODUCT ||--o{ CART_ITEM : contained
    PRODUCT ||--|| PRODUCT_STAT : summarized_by

    ORDERS ||--o{ ORDER_ITEM : contains
    ORDERS ||--|| PAYMENT : paid_by
    ORDERS ||--|| DELIVERY : delivered_by
    ORDERS ||--o{ ORDER_EVENT_LOG : logs

    ORDER_ITEM ||--o| REVIEW : reviewed_by

    CART ||--o{ CART_ITEM : contains

    COUPON ||--o{ COUPON_ISSUE : issued_as
    ORDERS ||--o| COUPON_ISSUE : uses
```

```
BATCH_JOB_EXECUTION ||--o{ BATCH_FAILURE : has
```

2. 핵심 설계 원칙

2-1. 주문과 상품은 다대다로 설계하지 않는다

주문과 상품은 겉으로는 N:M 관계처럼 보인다.

```
Order N : M Product
```

하지만 실무에서는 반드시 중간 엔티티인 `order_item` 을 둔다.

```
orders 1 : N order_item
product 1 : N order_item
```

이유는 주문상품에 단순 연결 정보만 있는 것이 아니라, 주문 당시의 상품 정보가 저장되어야 하기 때문이다.

- 주문 당시 상품명
- 주문 당시 가격
- 주문 수량
- 주문 당시 옵션명
- 할인 금액

상품명이 바뀌어도 과거 주문 내역은 바뀌면 안 된다.
상품 가격이 바뀌어도 과거 주문 금액은 바뀌면 안 된다.

2-2. 상품과 재고를 분리한다

상품 정보와 재고 정보는 변경 성격이 다르다.

```
product
- 이름, 가격, 카테고리, 판매 상태

inventory
- 총 재고, 예약 재고, 판매 가능 재고
```

상품 정보는 조회가 많고 변경이 상대적으로 적다.
재고 정보는 주문 시마다 자주 변경되며 동시성 제어가 필요하다.

따라서 운영 환경에서는 `product.stock_quantity` 하나로 처리하기보다 `inventory` 테이블을 분리하는 것이 좋다.

2-3. 결제는 멍등성을 가진다

결제 승인 요청과 PG 콜백은 중복으로 들어올 수 있다.

따라서 다음 값은 UNIQUE 제약조건을 둔다.

```
payment_key
idempotency_key
pg_transaction_id
```

동일 결제가 중복 요청되어도 한 번만 승인 처리되어야 한다.

2-4. 리뷰 중복 작성은 DB 제약조건으로 막는다

애플리케이션에서 먼저 조회하고 막는 것만으로는 부족하다.

```
A 요청: 리뷰 없음 확인
B 요청: 리뷰 없음 확인
A 요청: 리뷰 INSERT
B 요청: 리뷰 INSERT
```

따라서 `review.order_item_id` 에 UNIQUE 제약조건을 둔다.

```
ALTER TABLE review
ADD CONSTRAINT uk_review_order_item UNIQUE(order_item_id);
```

3. 테이블 상세 설계

3-1. member

회원 테이블이다.

```
CREATE TABLE member (
  member_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  email VARCHAR(255) NOT NULL,
```

```
password VARCHAR(255) NOT NULL,
name VARCHAR(100) NOT NULL,
phone VARCHAR(30),
role VARCHAR(30) NOT NULL,
status VARCHAR(30) NOT NULL,
created_at DATETIME NOT NULL,
updated_at DATETIME NOT NULL,
deleted_at DATETIME NULL,

CONSTRAINT uk_member_email UNIQUE(email)
);
```

상태값

```
role
- USER
- ADMIN

status
- ACTIVE
- DORMANT
- WITHDRAWN
- BLOCKED
```

주요 인덱스

```
CREATE INDEX ix_member_status_created
ON member(status, created_at DESC);
```

3-2. category

상품 카테고리 테이블이다.

```
CREATE TABLE category (
category_id BIGINT PRIMARY KEY AUTO_INCREMENT,
parent_id BIGINT NULL,
name VARCHAR(100) NOT NULL,
depth INT NOT NULL,
sort_order INT NOT NULL,
is_active BOOLEAN NOT NULL,
created_at DATETIME NOT NULL,
updated_at DATETIME NOT NULL,
```

```
CONSTRAINT fk_category_parent
    FOREIGN KEY(parent_id) REFERENCES category(category_id)
);
```

주요 인덱스

```
CREATE INDEX ix_category_parent_sort
ON category(parent_id, sort_order);
```

3-3. product

상품 기본 정보 테이블이다.

```
CREATE TABLE product (
    product_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    category_id BIGINT NOT NULL,
    name VARCHAR(255) NOT NULL,
    price INT NOT NULL,
    status VARCHAR(30) NOT NULL,
    thumbnail_url VARCHAR(500),
    description TEXT,
    created_at DATETIME NOT NULL,
    updated_at DATETIME NOT NULL,
    deleted_at DATETIME NULL,

    CONSTRAINT fk_product_category
        FOREIGN KEY(category_id) REFERENCES category(category_id),

    CONSTRAINT chk_product_price
        CHECK(price >= 0)
);
```

상태값

```
status
- SELLING
- SOLD_OUT
- HIDDEN
- DISCONTINUED
```

주요 인덱스

상품 목록 조회용:

```
CREATE INDEX ix_product_category_status_created
ON product(category_id, status, created_at DESC, product_id DESC);
```

관리자 상품 검색용:

```
CREATE INDEX ix_product_status_created
ON product(status, created_at DESC, product_id DESC);
```

3-4. inventory

재고 테이블이다.

```
CREATE TABLE inventory (
  product_id BIGINT PRIMARY KEY,
  total_quantity INT NOT NULL,
  reserved_quantity INT NOT NULL,
  available_quantity INT NOT NULL,
  version BIGINT NOT NULL,
  updated_at DATETIME NOT NULL,

  CONSTRAINT fk_inventory_product
    FOREIGN KEY(product_id) REFERENCES product(product_id),

  CONSTRAINT chk_inventory_quantity
    CHECK (
      total_quantity >= 0
      AND reserved_quantity >= 0
      AND available_quantity >= 0
    )
);
```

컬럼 의미

컬럼	의미
total_quantity	전체 보유 재고
reserved_quantity	결제 대기 등으로 예약된 재고
available_quantity	실제 주문 가능한 재고
version	낙관적 락용 버전

재고 예약 SQL

```
UPDATE inventory
SET reserved_quantity = reserved_quantity + :quantity,
    available_quantity = available_quantity - :quantity
WHERE product_id = :productId
AND available_quantity >= :quantity;
```

예약 재고 확정 SQL

```
UPDATE inventory
SET reserved_quantity = reserved_quantity - :quantity,
    total_quantity = total_quantity - :quantity
WHERE product_id = :productId
AND reserved_quantity >= :quantity;
```

예약 재고 해제 SQL

```
UPDATE inventory
SET reserved_quantity = reserved_quantity - :quantity,
    available_quantity = available_quantity + :quantity
WHERE product_id = :productId
AND reserved_quantity >= :quantity;
```

3-5. orders

주문 마스터 테이블이다.

```
CREATE TABLE orders (
    order_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    order_no VARCHAR(50) NOT NULL,
    member_id BIGINT NOT NULL,
    order_status VARCHAR(50) NOT NULL,
    total_price INT NOT NULL,
    total_discount_amount INT NOT NULL DEFAULT 0,
    final_payment_amount INT NOT NULL,
    ordered_at DATETIME NOT NULL,
    paid_at DATETIME NULL,
    cancelled_at DATETIME NULL,
    created_at DATETIME NOT NULL,
    updated_at DATETIME NOT NULL,

    CONSTRAINT fk_orders_member
```

```
FOREIGN KEY(member_id) REFERENCES member(member_id),

CONSTRAINT uk_orders_order_no UNIQUE(order_no),

CONSTRAINT chk_orders_price
CHECK(total_price >= 0 AND final_payment_amount >= 0)
);
```

주문 상태

```
PENDING_PAYMENT
PAID
PAYMENT_FAILED
PREPARING
SHIPPED
DELIVERED
CONFIRMED
CANCEL_REQUESTED
CANCELLED
REFUND_REQUESTED
REFUNDED
```

주요 인덱스

내 주문 목록 조회:

```
CREATE INDEX ix_orders_member_ordered
ON orders(member_id, ordered_at DESC, order_id DESC);
```

관리자 주문 검색:

```
CREATE INDEX ix_orders_status_ordered
ON orders(order_status, ordered_at DESC, order_id DESC);
```

결제 대기 만료 배치:

```
CREATE INDEX ix_orders_status_created
ON orders(order_status, created_at);
```

3-6. order_item

주문상품 테이블이다.


```

CREATE TABLE order_item (
  order_item_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT NOT NULL,
  product_id BIGINT NOT NULL,
  product_name_snapshot VARCHAR(255) NOT NULL,
  option_name_snapshot VARCHAR(255),
  order_price INT NOT NULL,
  quantity INT NOT NULL,
  discount_amount INT NOT NULL DEFAULT 0,
  created_at DATETIME NOT NULL,

  CONSTRAINT fk_order_item_order
    FOREIGN KEY(order_id) REFERENCES orders(order_id),

  CONSTRAINT fk_order_item_product
    FOREIGN KEY(product_id) REFERENCES product(product_id),

  CONSTRAINT chk_order_item_quantity
    CHECK(quantity > 0),

  CONSTRAINT chk_order_item_price
    CHECK(order_price >= 0)
);

```

주요 인덱스

```

CREATE INDEX ix_order_item_order
ON order_item(order_id);

CREATE INDEX ix_order_item_product
ON order_item(product_id);

```

3-7. payment

결제 테이블이다.

```

CREATE TABLE payment (
  payment_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT NOT NULL,
  payment_key VARCHAR(100) NOT NULL,
  pg_provider VARCHAR(50) NOT NULL,
  pg_transaction_id VARCHAR(100),
  idempotency_key VARCHAR(150) NOT NULL,
  payment_status VARCHAR(50) NOT NULL,

```

```

requested_amount INT NOT NULL,
approved_amount INT NOT NULL DEFAULT 0,
cancelled_amount INT NOT NULL DEFAULT 0,
approved_at DATETIME NULL,
failed_at DATETIME NULL,
cancelled_at DATETIME NULL,
failure_code VARCHAR(100),
failure_message VARCHAR(500),
created_at DATETIME NOT NULL,
updated_at DATETIME NOT NULL,

CONSTRAINT fk_payment_order
    FOREIGN KEY(order_id) REFERENCES orders(order_id),

CONSTRAINT uk_payment_key UNIQUE(payment_key),
CONSTRAINT uk_payment_idempotency_key UNIQUE(idempotency_key),
CONSTRAINT uk_payment_pg_transaction UNIQUE(pg_transaction_id),

CONSTRAINT chk_payment_amount
    CHECK(requested_amount >= 0 AND approved_amount >= 0 AND cancelled_amount >= 0)
);

```

결제 상태

```

READY
REQUESTED
APPROVED
FAILED
CANCELLED
PARTIAL_CANCELLED
REFUNDED

```

주요 인덱스

```

CREATE INDEX ix_payment_order
ON payment(order_id);

CREATE INDEX ix_payment_status_created
ON payment(payment_status, created_at);

CREATE INDEX ix_payment_approved_order_status
ON payment(payment_status, approved_at);

```

3-8. delivery

배송 테이블이다.

```
CREATE TABLE delivery (  
  delivery_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  order_id BIGINT NOT NULL,  
  receiver_name VARCHAR(100) NOT NULL,  
  receiver_phone VARCHAR(30) NOT NULL,  
  zipcode VARCHAR(20) NOT NULL,  
  address1 VARCHAR(255) NOT NULL,  
  address2 VARCHAR(255),  
  delivery_status VARCHAR(50) NOT NULL,  
  tracking_number VARCHAR(100),  
  shipped_at DATETIME NULL,  
  delivered_at DATETIME NULL,  
  created_at DATETIME NOT NULL,  
  updated_at DATETIME NOT NULL,  
  
  CONSTRAINT fk_delivery_order  
    FOREIGN KEY(order_id) REFERENCES orders(order_id),  
  
  CONSTRAINT uk_delivery_order UNIQUE(order_id)  
);
```

배송 상태

```
READY  
PREPARING  
SHIPPED  
DELIVERED  
RETURNING  
RETURNED
```

주요 인덱스

```
CREATE INDEX ix_delivery_status_created  
ON delivery(delivery_status, created_at);
```

3-9. review

리뷰 테이블이다.

```

CREATE TABLE review (
  review_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  member_id BIGINT NOT NULL,
  product_id BIGINT NOT NULL,
  order_item_id BIGINT NOT NULL,
  rating INT NOT NULL,
  content TEXT NOT NULL,
  created_at DATETIME NOT NULL,
  updated_at DATETIME NOT NULL,
  deleted_at DATETIME NULL,

  CONSTRAINT fk_review_member
    FOREIGN KEY(member_id) REFERENCES member(member_id),

  CONSTRAINT fk_review_product
    FOREIGN KEY(product_id) REFERENCES product(product_id),

  CONSTRAINT fk_review_order_item
    FOREIGN KEY(order_item_id) REFERENCES order_item(order_item_id),

  CONSTRAINT uk_review_order_item UNIQUE(order_item_id),

  CONSTRAINT chk_review_rating
    CHECK(rating BETWEEN 1 AND 5)
);

```

주요 인덱스

상품 리뷰 목록:

```

CREATE INDEX ix_review_product_created
ON review(product_id, created_at DESC, review_id DESC);

```

내 리뷰 목록:

```

CREATE INDEX ix_review_member_created
ON review(member_id, created_at DESC, review_id DESC);

```

3-10. product_stat

상품 통계 테이블이다.

```

CREATE TABLE product_stat (
  product_id BIGINT PRIMARY KEY,
  review_count INT NOT NULL DEFAULT 0,
  average_rating DECIMAL(3,2) NOT NULL DEFAULT 0,
  rating_1_count INT NOT NULL DEFAULT 0,
  rating_2_count INT NOT NULL DEFAULT 0,
  rating_3_count INT NOT NULL DEFAULT 0,
  rating_4_count INT NOT NULL DEFAULT 0,
  rating_5_count INT NOT NULL DEFAULT 0,
  sales_count INT NOT NULL DEFAULT 0,
  updated_at DATETIME NOT NULL,

  CONSTRAINT fk_product_stat_product
    FOREIGN KEY(product_id) REFERENCES product(product_id)
);

```

3-11. cart

장바구니 테이블이다.

```

CREATE TABLE cart (
  cart_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  member_id BIGINT NOT NULL,
  created_at DATETIME NOT NULL,
  updated_at DATETIME NOT NULL,

  CONSTRAINT fk_cart_member
    FOREIGN KEY(member_id) REFERENCES member(member_id),

  CONSTRAINT uk_cart_member UNIQUE(member_id)
);

```

3-12. cart_item

장바구니 상품 테이블이다.

```

CREATE TABLE cart_item (
  cart_item_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  cart_id BIGINT NOT NULL,
  product_id BIGINT NOT NULL,
  quantity INT NOT NULL,
  created_at DATETIME NOT NULL,

```

```

updated_at DATETIME NOT NULL,

CONSTRAINT fk_cart_item_cart
    FOREIGN KEY(cart_id) REFERENCES cart(cart_id),

CONSTRAINT fk_cart_item_product
    FOREIGN KEY(product_id) REFERENCES product(product_id),

CONSTRAINT uk_cart_item_product UNIQUE(cart_id, product_id),

CONSTRAINT chk_cart_item_quantity
    CHECK(quantity > 0)
);

```

3-13. coupon

쿠폰 마스터 테이블이다.

```

CREATE TABLE coupon (
    coupon_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    discount_type VARCHAR(30) NOT NULL,
    discount_value INT NOT NULL,
    minimum_order_amount INT NOT NULL DEFAULT 0,
    issued_quantity INT NOT NULL,
    used_quantity INT NOT NULL DEFAULT 0,
    valid_from DATETIME NOT NULL,
    valid_to DATETIME NOT NULL,
    status VARCHAR(30) NOT NULL,
    created_at DATETIME NOT NULL,
    updated_at DATETIME NOT NULL,

    CONSTRAINT chk_coupon_quantity
        CHECK(issued_quantity >= 0 AND used_quantity >= 0)
);

```

3-14. coupon_issue

회원에게 발급된 쿠폰 테이블이다.

```

CREATE TABLE coupon_issue (
    coupon_issue_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    coupon_id BIGINT NOT NULL,

```

```

member_id BIGINT NOT NULL,
order_id BIGINT NULL,
status VARCHAR(30) NOT NULL,
issued_at DATETIME NOT NULL,
used_at DATETIME NULL,
expired_at DATETIME NULL,

CONSTRAINT fk_coupon_issue_coupon
    FOREIGN KEY(coupon_id) REFERENCES coupon(coupon_id),

CONSTRAINT fk_coupon_issue_member
    FOREIGN KEY(member_id) REFERENCES member(member_id),

CONSTRAINT fk_coupon_issue_order
    FOREIGN KEY(order_id) REFERENCES orders(order_id)
);

```

주요 제약조건

동일 쿠폰을 회원이 한 번만 받을 수 있게 하려면 다음 제약조건을 둔다.

```

CREATE UNIQUE INDEX uk_coupon_issue_member_coupon
ON coupon_issue(member_id, coupon_id);

```

3-15. point_history

포인트 이력 테이블이다.

```

CREATE TABLE point_history (
    point_history_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    member_id BIGINT NOT NULL,
    order_id BIGINT NULL,
    point_type VARCHAR(30) NOT NULL,
    amount INT NOT NULL,
    balance_after INT NOT NULL,
    reason VARCHAR(255) NOT NULL,
    created_at DATETIME NOT NULL,

    CONSTRAINT fk_point_history_member
        FOREIGN KEY(member_id) REFERENCES member(member_id),

    CONSTRAINT fk_point_history_order

```

```
FOREIGN KEY(order_id) REFERENCES orders(order_id)
);
```

포인트 타입

```
EARN
USE
CANCEL_EARN
CANCEL_USE
EXPIRE
```

3-16. outbox_event

이벤트 발행 보장용 Outbox 테이블이다.

```
CREATE TABLE outbox_event (
  event_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  aggregate_type VARCHAR(100) NOT NULL,
  aggregate_id BIGINT NOT NULL,
  event_type VARCHAR(100) NOT NULL,
  payload JSON NOT NULL,
  status VARCHAR(30) NOT NULL,
  retry_count INT NOT NULL DEFAULT 0,
  created_at DATETIME NOT NULL,
  published_at DATETIME NULL
);
```

주요 인덱스

```
CREATE INDEX ix_outbox_status_created
ON outbox_event(status, created_at);
```

3-17. order_event_log

주문 상태 변경 이력 테이블이다.

```
CREATE TABLE order_event_log (
  order_event_log_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT NOT NULL,
  from_status VARCHAR(50),
  to_status VARCHAR(50) NOT NULL,
```



```

event_type VARCHAR(100) NOT NULL,
reason VARCHAR(500),
created_by BIGINT NULL,
created_at DATETIME NOT NULL,

CONSTRAINT fk_order_event_log_order
    FOREIGN KEY(order_id) REFERENCES orders(order_id)
);

```

주요 인덱스

```

CREATE INDEX ix_order_event_log_order_created
ON order_event_log(order_id, created_at);

```

3-18. batch_job_execution

배치 실행 이력 테이블이다.

```

CREATE TABLE batch_job_execution (
    execution_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    job_name VARCHAR(100) NOT NULL,
    status VARCHAR(30) NOT NULL,
    started_at DATETIME NOT NULL,
    ended_at DATETIME NULL,
    processed_count INT NOT NULL DEFAULT 0,
    success_count INT NOT NULL DEFAULT 0,
    fail_count INT NOT NULL DEFAULT 0,
    last_processed_id BIGINT NULL,
    error_message TEXT NULL
);

```

3-19. batch_failure

배치 실패 데이터 테이블이다.

```

CREATE TABLE batch_failure (
    failure_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    execution_id BIGINT NOT NULL,
    target_id BIGINT NOT NULL,
    reason VARCHAR(500) NOT NULL,
    payload JSON NULL,
    created_at DATETIME NOT NULL,
);

```

```
CONSTRAINT fk_batch_failure_execution
FOREIGN KEY(execution_id) REFERENCES batch_job_execution(execution_id)
);
```

4. 인덱스 설계 요약

4-1. 사용자 화면 조회용

```
CREATE INDEX ix_product_category_status_created
ON product(category_id, status, created_at DESC, product_id DESC);

CREATE INDEX ix_orders_member_ordered
ON orders(member_id, ordered_at DESC, order_id DESC);

CREATE INDEX ix_review_product_created
ON review(product_id, created_at DESC, review_id DESC);
```

4-2. 관리자 검색용

```
CREATE INDEX ix_orders_status_ordered
ON orders(order_status, ordered_at DESC, order_id DESC);

CREATE INDEX ix_product_status_created
ON product(status, created_at DESC, product_id DESC);

CREATE INDEX ix_payment_status_created
ON payment(payment_status, created_at);
```

4-3. 배치 처리용

```
CREATE INDEX ix_orders_status_created
ON orders(order_status, created_at);

CREATE INDEX ix_outbox_status_created
ON outbox_event(status, created_at);
```

```
CREATE INDEX ix_order_event_log_order_created
ON order_event_log(order_id, created_at);
```

5. 설계상 중요한 제약조건 요약

```
-- 회원 이메일 중복 방지
ALTER TABLE member
ADD CONSTRAINT uk_member_email UNIQUE(email);

-- 주문번호 중복 방지
ALTER TABLE orders
ADD CONSTRAINT uk_orders_order_no UNIQUE(order_no);

-- 결제키 중복 방지
ALTER TABLE payment
ADD CONSTRAINT uk_payment_key UNIQUE(payment_key);

-- 결제 역등기 중복 방지
ALTER TABLE payment
ADD CONSTRAINT uk_payment_idempotency_key UNIQUE(idempotency_key);

-- 리뷰 중복 작성 방지
ALTER TABLE review
ADD CONSTRAINT uk_review_order_item UNIQUE(order_item_id);

-- 장바구니 동일 상품 중복 방지
ALTER TABLE cart_item
ADD CONSTRAINT uk_cart_item_product UNIQUE(cart_id, product_id);
```

6. 최종 정리

이 ERD의 핵심은 다음과 같다.

1. 주문과 상품은 OrderItem으로 풀어낸다.
2. 주문 당시 상품명, 가격, 옵션은 스냅샷으로 저장한다.
3. 상품과 재고는 분리한다.
4. 재고는 available_quantity, reserved_quantity, total_quantity로 관리한다.
5. 결제는 payment_key와 idempotency_key로 중복 처리를 방지한다.
6. 리뷰 중복은 DB UNIQUE 제약조건으로 막는다.

7. 조회 패턴 기준으로 인덱스를 설계한다.
8. 장애 복구를 위해 `outbox_event`, `order_event_log`, `batch_execution` 테이블을 둔다.

운영 가능한 쇼핑몰 ERD는 단순 관계도보다 **정합성, 중복 방지, 장애 복구, 조회 성능**을 함께 담아야 한다.